

Cuckoo Trie Vectorization

Gal Shwed

Ido Moran

Liran Napadenski

Abstract

The Cuckoo Trie is a high-performance ordered in-memory index that achieves high throughput through a bucketized cuckoo hash table and prefix-hashing lookups, specifically designed to address memory dependency bottlenecks. Commonly employed in scenarios requiring fast sequence retrieval and memory-level parallelism, it serves as a critical infrastructure component in modern data systems. In this study, we investigate the impact of Single Instruction, Multiple Data (SIMD) vectorization on the Cuckoo Trie’s search paths, with a focus on its 4-cell bucket search helpers. Our objective was to evaluate whether manual AVX2 vectorization could reduce per-lookup instruction overhead and improve throughput without compromising the trie’s memory efficiency.

Through a series of optimization attempts—including SIMD header matching, sidecar prefiltering, and adaptive dispatch—we find that the vectorization experiment largely favored scalar execution for latency-sensitive single-threaded lookups. In the vast majority of our tested benchmark configurations, the irregular structure and significant early-exit potential of point-lookup bucket searches negated the benefits of vectorization. However, these results do show strong promise when looking at high-contention environments: the subset of benchmarks that did yield performance gains occurred entirely under high-throughput, multi-threaded workloads, where SIMD’s fixed overhead is effectively amortized. These results highlight a fundamental tradeoff, suggesting that future gains lie in batch-oriented parallelism rather than fine-grained intra-lookup vectorization.

1 Introduction

Modern data systems increasingly depend on in-memory indexing for low-latency request handling, stream processing, and transactional workloads where both high throughput and predictable tail behavior matter. In these settings, the index is often a central performance limiter rather than a peripheral data structure, especially when workloads mix reads, inserts, and ordered operations. For this reason, the practical question is not only whether an index is asymptotically efficient, but whether its implementation maps well to contemporary CPU and memory behavior.

A key practical constraint is the gap between processor speed and memory access cost. Even when an operation performs relatively little arithmetic, throughput can collapse if execution repeatedly waits on dependent cache misses. From a systems perspective, this makes memory-level parallelism (MLP)¹ a first-order concern: high-performance index implementations must expose independent memory accesses and minimize strictly serial memory dependencies whenever possible. The Cuckoo Trie is an instructive case study because it was explicitly designed around these realities, combining ordered indexing with mechanisms intended to exploit MLP in DRAM-resident workloads (Zeitak and Morrison, 2021).

This project focuses on vectorization-aware optimization of the Cuckoo Trie implementation. Concretely, our goal is to identify hot execution paths in the provided codebase, determine where Single Instruction, Multiple Data (SIMD)² operations can reduce per-operation

¹Memory-level parallelism refers to the ability to have multiple pending memory operations in flight simultaneously, an essential mitigation for the “memory wall” (Wulf and McKee, 1995).

²SIMD architectures, such as x86 AVX2 and SSE,

overhead or improve effective instruction-level throughput, and implement changes that preserve correctness while improving measured performance. Unlike traditional trie structures that rely on pointer-chasing³, the Cuckoo Trie replaces explicit node pointers with a bucketized layout, offering a uniquely regular structure that is highly amenable to batch processing and SIMD techniques. This sets it apart from typical indexing structures and positions it as a prime candidate for memory-level parallelism improvements.

The complete source code and reproducible benchmark framework for this study are publicly available at <https://github.com/gal-shwed/cuckoo-trie-vectorize>.

2 Background

2.1 Cuckoo Trie Principles

The Cuckoo Trie (Zeitak and Morrison, 2021) is a high-performance ordered index designed to overcome the pointer-chasing bottlenecks of traditional tries. By replacing explicit node pointers with a hashed bucketized representation, it enables independent, prefetched memory accesses, thereby maximizing memory-level parallelism (MLP). We refer the reader to the original paper for detailed discussions on node layouts, path compression, and the “key elimination” mechanism that ensures compact memory footprints (Zeitak and Morrison, 2021).

2.2 SIMD Vectorization

Vectorization utilizes Single Instruction, Multiple Data (SIMD) instructions to process multiple data elements concurrently within dedicated wide registers (e.g., AVX2 and SSE). For x86 systems, these instructions can significantly multiply throughput for regular, data-parallel patterns (Intel Corporation, 2026a). However, irregular control flow and early-exit opportunities—both common in search-intensive algorithms—can negate the benefits of manual vectorization, necessitating empirical verification across target workloads (Kim et al., 2010; Free Software Foundation, 2026).

allow operations on vectors of data simultaneously, enhancing instruction-level throughput (Intel Corporation, 2026b).

³Pointer-chasing refers to the serial dependence of node addresses, which limits the effectiveness of hardware prefetchers.

2.3 The Scalar-SIMD Tradeoff

Modern out-of-order processors excel at executing instructions as dependencies allow, but remain sensitive to memory latency. When a search loop processes a fixed-width array of records (such as a four-cell bucket), theoretically, SIMD can vectorize the comparison of all elements. However, scalar execution often benefits from early-exit behavior, terminating as soon as a match is found. The core tradeoff is whether the overhead of SIMD setup and enforced fixed-width work is justified by the reduction in raw comparison operations.

3 Experimental Methodology

We approached the evaluation as a controlled measurement problem, treating each optimization as an empirical hypothesis. Our protocol combined full-suite throughput comparisons, minimal diagnostic gates to reject weak variants quickly, and targeted instrumentation to understand microarchitectural behavior.

3.1 Experimental Setup

All experiments were conducted on the TAU lab environment using an x86_64 Linux machine (rack-mad-22r). The environment details are critical as the codebase is sensitive to memory placement and huge-page configuration. Table 1 summarizes the main configuration choices.

Table 1: Experimental setup summary.

Component	Details
Platform	Linux x86_64, TAU Rack-MAD-22R.
Build	GCC, <code>-march=haswell</code> , optimizations enabled.
Runtime	<code>numactl -interleave=all</code> , <code>jemalloc huge-pages</code> .
Benchmarks	YCSB A–F, 1 and 24 threads, 5 repetitions per run.

3.2 Optimization Strategy

While the Cuckoo Trie is designed for memory-level parallelism, we hypothesized that scalar hot paths in the bucket search logic could benefit from Single Instruction, Multiple Data (SIMD) vectorization. Each trie lookup involves scanning a four-cell bucket and testing compact header predicates. We primarily focused on vectorizing these bucket

search helpers mentioned in the project brief: `find_entry_in_bucket_by_color()` and `find_entry_in_bucket_by_parent()`. For each attempt, we measured the throughput change on `ycsb-c` (100% read) and `ycsb-e` (scan-heavy) as a primary filter.

We treat the project as a sequence of optimization hypotheses. In each lookup, the trie first materializes symbols, descends path-compressed jump nodes, and finally searches the candidate hash-table bucket. Our work targets this final search stage by exploring multiple avenues to integrate parallelized header comparisons within the narrow (4-cell) search unit.

4 Implementation Attempts

We evaluated several implementation variants targeting different parts of the trie search path. This section outlines our progressive attempts to vectorize core operations.

4.1 Attempt 1: Color-Path AVX2 Match

We replaced the scalar header loop in `find_entry_in_bucket_by_color()` with a 4-lane AVX2 search. The implementation masks headers and compares against target values using `_mm256_cmpeq_epi64`, loading full entries only on matching lanes.

4.2 Attempt 2: Parent-Path AVX2 Match

We applied a similar SIMD strategy to `find_entry_in_bucket_by_parent()`, a much hotter path during trie traversal. This parallelizes comparisons over parent colors and tags across the 4-cell bucket.

4.3 Attempt 3: Sidecar Signature Prefilter

To reduce memory operations, we added a 1-byte signature sidecar per bucket. Lookups first apply an AVX2 prefilter over all four signatures collectively, aiming to avoid unnecessary header accesses.

4.4 Attempt 4 & 5: Advanced Control Flow (Two-Stage & Adaptive)

We introduced two dynamic approaches: splitting vectorization into two stages (lanes 0-1

then 2-3) to exploit early matches, and building an adaptive dynamic dispatch that toggles between AVX2 and scalar paths based on observed hit-ratios.

4.5 Attempt C1: Key-Symbol SIMD Fast Path

Finally, we optimized `key_to_symbols()`, a computation-heavy stage. For 8-byte keys, we designed an AVX2 fast-path to process two 40-bit chunks in parallel, eliminating the scalar transformation sequence.

5 Datasets and Evaluation

We evaluated our implementations using the project’s standard benchmarking workflow, focusing on controlled random-key workloads.

5.1 Dataset: rand-8

The primary dataset used throughout this study is **rand-8**, which consists of random 8-byte keys generated with a fixed seed (Seed 1). This workload represents a common point-lookup scenario for in-memory indexes and is particularly challenging for ordered structures that must balance compaction with search efficiency.

5.2 YCSB Benchmark Suite

We used the Yahoo Cloud Serving Benchmark (YCSB) to simulate various production workloads: - **YCSB-C (100% Read)**: Our primary lookup-path performance indicator, used to isolate the effect of SIMD on read-only trails. - **YCSB-E (95% Scan, 5% Insert)**: A scan-heavy workload that exposes traversal-side regressions that pure point-lookup benchmarks might miss. - **YCSB-A/B/D/F**: Mixed workloads including updates, inserts, and read-modify-write operations, used for full-suite verification.

5.3 Evaluation Metrics

We reported outcomes using the following metrics collected via hardware performance counters: - **Throughput (Mops/s)**: The core measure of operation-level performance. - **Cycles/op**: Total CPU cycles consumed per request, reflecting instruction-level overhead. - **Instructions/op**: The instruction count per operation, used to verify whether SIMD effectively reduced the number of retired in-

structions. - **Branch and Cache Statistics:** Specifically Branch Misses and L1-Miss/op, which helped identify microarchitectural efficiency trade-offs.

Benchmarks were executed in both single-threaded (1 thread) and highly concurrent (24 threads) configurations. Each configuration was repeated 5 times, and results were summarized by the mean throughput across repetitions.

6 Results

Across all evaluated variants, the project was fundamentally a failure for single-threaded point lookups. Among all scenarios tested, only 2 out of 14 benchmark configurations demonstrated any performance improvement. However, these successes do show promise for specific environments: **Attempt 2 (Parent-Path SIMD)** and **Attempt C1 (Key-Symbol SIMD)** emerged as the most effective optimizations. These results reveal a clear tradeoff: SIMD severely harms latency in single-threaded lookups due to early-exit behavior, but provides measurable, promising gains under high-throughput, multi-threaded workloads where contention is high.

6.1 Performance Summary of Top Attempts

Table 2 summarizes the throughput deltas for Attempt 2 and Attempt C1 in both their strongest and weakest scenarios. A complete breakdown of all YCSB A–F results, including single- and multi-threaded configurations, is provided in Appendix A.

Table 2: Throughput delta (%) for the two most successful attempts.

Benchmark	Attempt 2	Attempt C1
mt-ycsb-f (24-thread)	+16.4%	+5.3%
mt-ycsb-a (24-thread)	+11.6%	+1.7%
ycsb-c (1-thread)	-22.0%	-8.0%
ycsb-f (1-thread)	-25.9%	-7.2%

6.2 Deep Dive into Multi-Threaded Performance

The observed speedups in **mt-ycsb-f** (read-modify-write) and **mt-ycsb-a** (50/50 read-write) suggest that the fixed overhead of SIMD setup is more effectively amortized under high contention. In such environments, the CPU

becomes bottlenecked by execution throughput rather than strict instruction latency. Because SIMD bundles four lane-checks into a single hardware operation, it significantly decreases the total number of instructions the CPU must execute per lookup, freeing up hardware processing capacity. Furthermore, the CPU’s out-of-order execution engine can overlap independent instruction streams from multiple threads, effectively hiding that SIMD setup latency.

In contrast, for the single-threaded, read-only workload **ycsb-c**, the execution remains on a tight critical path, where any additional setup cost directly impacts latency.

6.3 The “Early-Exit Tax” and Contention-Driven Distribution

To better understand the persistent regressions in single-threaded workloads and the divergence in multi-threaded ones, we analyzed the distribution of matches across the four lanes of the buckets. Table 3 shows the percentage of matches per lane.

Table 3: Primary and Secondary bucket lane hit distribution (%).

Workload	Bucket	Hit %	Lane 0	Lane 1	Lane 2	Lane 3
ycsb-c	Primary	98.1	84.7	11.2	3.0	1.1
	Secondary	100.0	50.6	30.2	12.2	7.0
mt-ycsb-a	Primary	97.1	82.5	12.1	3.7	1.6
	Secondary	99.1	53.8	27.6	11.7	6.9
mt-ycsb-f	Primary	97.0	82.3	12.2	3.8	1.7
	Secondary	100.0	54.0	27.4	11.7	6.9
ycsb-f	Primary	97.8	84.1	11.5	3.2	1.3
	Secondary	100.0	51.7	29.2	12.0	7.0

In the latency-critical **ycsb-c** workload, over 93% of all matches occur within the first two lanes of the primary bucket. Scalar execution exploits this massive skew through immediate termination, whereas SIMD-based implementations typically incur a fixed cost to evaluate all four lanes regardless.

However, in successful workloads like **mt-ycsb-f** and **mt-ycsb-a**, the presence of heavy updates and inserts introduces contention and bucket hash collisions. This fundamentally alters the search path: lookups overflow much more frequently into the secondary bucket. More importantly, matches within this secondary bucket are far more evenly distributed across all slots (with Lane 0 hits dropping to ~53%-54%, while deeper Lanes 1–3 account for the remaining ~46%-47%).

This effect highlights the core tradeoff, which we term the **early-exit tax**. When a highly

skewed primary distribution strictly favors instantaneous scalar termination, SIMD lookups fail to outperform the baseline. However, when high contention and overflow force lookups repeatedly into deeper secondary lanes, the simultaneous multi-lane evaluation of SIMD flips from a latency penalty into a distinct throughput advantage. While SIMD improves overall efficiency in bulk-oriented, high-contention environments, pure read-only point lookups remain inherently favorable to scalar execution.

When comparing our two most successful implementations, this tradeoff dictates their viability: Attempt C1 targets `key_to_symbols()`, a purely computational stage that is completely isolated from bucket-level search lanes. As a result, it is largely immune to the *early-exit tax*. Its performance is almost solely dependent on high-throughput, multi-threaded contention to amortize its fixed register setup overhead, making its single-threaded regressions much milder overall. Overall, Attempt C1 proves to be much more stable across the diverse benchmark suite. When averaging the speedups across all 14 evaluated configurations, Attempt C1 achieves a mean speedup of $0.96\times$, significantly outperforming Attempt 2’s highly volatile mean of $0.89\times$.

Conversely, Attempt 2 vectorizes the actual bucket searches, marrying its effectiveness directly to the early-exit tax. When searches are highly skewed toward the first lane (low miss rate), Attempt 2 suffers significantly. However, when higher miss rates consistently push lookups into evenly distributed secondary buckets, Attempt 2 maximizes SIMD lane utilization and dramatically outperforms Attempt C1 (achieving peak speedups of +16.4% compared to C1’s +5.3%).

7 Conclusion and Future Work

Our study of the Cuckoo Trie search path indicates that, under its current bucketized architecture, in-request SIMD vectorization of individual 4-cell lookups is largely ineffective for latency-sensitive workloads. The primary limitation is the overhead of SIMD register setup relative to the small amount of useful work per lookup, further amplified by the high early-exit rate (exceeding 80% in workloads such as `ycsb-c`). In such cases, scalar execu-

tion avoids unnecessary work and remains more efficient on the critical path.

At the same time, our results demonstrate that SIMD vectorization can provide meaningful performance improvements under high-throughput, multi-threaded workloads. In these scenarios, the fixed cost of SIMD setup is effectively amortized across increased contention and parallel activity. Specifically, we observe that computational vectorization (such as Key-Symbol SIMD) delivers stable but modest improvements by sidestepping search layout issues altogether. In contrast, search-path vectorization (such as Parent-Path SIMD) is highly sensitive to the *early-exit tax*: while it regresses heavily in highly skewed single-threaded lookups, it extracts the largest overall peak speedups when heavy contention and high miss rates flatten the search distributions.

Taken together, these findings highlight a fundamental tradeoff: SIMD is ill-suited for fine-grained, latency-critical search paths, but can be beneficial in throughput-oriented environments. This shifts the optimization focus away from microarchitectural improvements within a single lookup and toward higher-level structural or workload-driven approaches.

In particular, *batch lookup* parallelism emerges as a promising direction, where SIMD lanes are utilized across independent requests rather than within a single bucket. This approach better matches the strengths of SIMD execution and aligns with prior work on high-performance tree traversal (Kim et al., 2010), which emphasizes wide, batch-oriented parallelism as a key enabler of consistent speedups.

In summary, while the Cuckoo Trie’s prefix-hashing design exposes opportunities for memory-level parallelism, its internal search loops are already highly optimized in scalar form for low-latency execution. Achieving further performance improvements will likely require redefining the unit of parallelism—from intra-bucket operations to inter-request batching—thereby aligning SIMD execution with workload-level parallelism rather than fine-grained search logic.

A Comprehensive Results Table

Table 4 provides the complete throughput (Mops/s) and speedup results for the two most successful optimization attempts, Attempt 2 (Parent-Path SIMD) and Attempt C1 (Key-Symbol SIMD), compared across all standard YCSB workloads.

B Extended Lane Distribution Data

Tables 5 and 6 provide an extended view of the secondary and primary bucket hit distribution data across all benchmark workloads.

References

- Free Software Foundation. 2026. *Using the GNU Compiler Collection (GCC)*. Section on Option Summary and Optimize Options.
- Intel Corporation. 2026a. *Intel 64 and IA-32 Architectures Software Developer’s Manual*. Volume 2 (2A, 2B, 2C & 2D): Instruction Set Reference, A-Z.
- Intel Corporation. 2026b. *Intel Intrinsics Guide*. Interactive documentation for Intel C/C++ compiler intrinsics.
- Changkyu Kim, Jatin Chhugani, Nadathur Satish, Eric Sedlar, Anthony D. Nguyen, Tim Kaldewey, Victor W. Lee, Scott A. Brandt, and Pradeep Dubey. 2010. **FAST: Fast architecture sensitive tree search on modern CPUs and GPUs**. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, pages 339–350.
- Wm. A. Wulf and Sally A. McKee. 1995. Hitting the memory wall: implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23(1):20–24.
- Adar Zeitak and Adam Morrison. 2021. Cuckoo trie: Exploiting memory-level parallelism for efficient dram indexing. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*.

Table 4: Comparison of Throughput (Mops/s) and Speedup (S) across all YCSB workloads (Baseline, Attempt 2, and Attempt C1).

Workload	Baseline (Mops/s)	Attempt 2 (Mops/s)	S	Attempt C1 (Mops/s)	S
<i>Single-threaded (1 thread)</i>					
insert	2.27	2.05	$0.90\times$	2.22	$0.97\times$
ycsb-a	4.00	3.25	$0.81\times$	3.73	$0.93\times$
ycsb-b	4.43	3.48	$0.78\times$	4.08	$0.92\times$
ycsb-c	4.50	3.51	$0.78\times$	4.14	$0.92\times$
ycsb-d	4.30	3.45	$0.80\times$	3.99	$0.93\times$
ycsb-e	0.26	0.26	$0.98\times$	0.26	$0.99\times$
ycsb-f	3.47	2.57	$0.74\times$	3.22	$0.93\times$
<i>Multi-threaded (24 threads)</i>					
mt-insert	44.41	39.12	$0.88\times$	43.20	$0.97\times$
mt-ycsb-a	10.08	11.25	$1.12\times$	10.25	$1.02\times$
mt-ycsb-b	90.45	74.93	$0.83\times$	86.72	$0.96\times$
mt-ycsb-c	95.87	76.67	$0.80\times$	90.06	$0.94\times$
mt-ycsb-d	80.28	68.00	$0.85\times$	78.60	$0.98\times$
mt-ycsb-e	5.84	5.69	$0.97\times$	5.67	$0.97\times$
mt-ycsb-f	10.56	12.28	$1.16\times$	11.11	$1.05\times$

Benchmark	Pair Calls	Sec Searches	Sec Hits	Sec Miss	Hit %	Lane 0 %	Lane 1 %	Lane 2 %	Lane 3 %	Early %
ycsb-a	103148621	1961257	1961257	0	100.0	50.61	30.16	12.18	7.04	80.78
ycsb-b	103148621	1961257	1961257	0	100.0	50.61	30.16	12.18	7.04	80.78
ycsb-c	103148621	1961257	1961257	0	100.0	50.61	30.16	12.18	7.04	80.78
ycsb-d	99778898	2036116	2036116	0	100.0	27.41	38.50	20.47	13.62	65.91
ycsb-e	62167789	556561	556561	0	100.0	30.72	39.37	19.78	10.12	70.09
ycsb-f	129595057	2818275	2818275	0	100.0	51.74	29.23	12.03	7.00	80.97
mt-ycsb-a	390095604	11348067	11247281	100786	99.11	53.78	27.58	11.72	6.92	81.36
mt-ycsb-b	388853278	11230325	11230164	161	100.0	53.71	27.62	11.74	6.93	81.33
mt-ycsb-c	388840472	11229987	11229987	0	100.0	53.71	27.62	11.74	6.93	81.33
mt-ycsb-d	340577515	7672698	7672697	1	100.0	37.84	37.83	15.58	8.75	75.67
mt-ycsb-e	430885020	6103321	6103321	0	100.0	47.64	34.61	11.79	5.96	82.25
mt-ycsb-f	558965600	16736605	16735204	1401	99.99	53.96	27.42	11.70	6.92	81.38

Table 5: Extended secondary bucket statistics for all workloads.

Benchmark	Pair Calls	Pri Hits	Pri Miss	Hit %	Lane 0 %	Lane 1 %	Lane 2 %	Lane 3 %	Early %
ycsb-a	103148621	101187364	1961257	98.10	84.71	11.21	2.96	1.12	95.93
ycsb-b	103148621	101187364	1961257	98.10	84.71	11.21	2.96	1.12	95.93
ycsb-c	103148621	101187364	1961257	98.10	84.71	11.21	2.96	1.12	95.93
ycsb-d	99778898	97742782	2036116	97.96	82.46	11.46	4.22	1.86	93.92
ycsb-e	62167789	61611228	556561	99.10	85.17	11.12	2.97	0.73	96.30
ycsb-f	129595057	126776782	2818275	97.83	84.10	11.47	3.18	1.25	95.57
mt-ycsb-a	390095604	378848323	11348067	97.12	82.54	12.11	3.74	1.61	94.65
mt-ycsb-b	388853278	377623114	11230325	97.11	82.51	12.13	3.75	1.61	94.64
mt-ycsb-c	388840472	377610485	11229987	97.11	82.51	12.13	3.75	1.61	94.64
mt-ycsb-d	340577515	332904818	7672698	97.75	79.39	12.92	5.45	2.24	92.31
mt-ycsb-e	430885020	424781699	6103321	98.58	80.37	13.84	4.79	0.99	94.21
mt-ycsb-f	558965600	542230396	16736605	97.01	82.28	12.23	3.83	1.67	94.50

Table 6: Extended primary bucket statistics for all workloads.